

AUTENTICACION BASADA EN TOKENS

¿Qué es la autenticación basada en token?

La autenticación basada en tokens es el proceso de verificar la identidad mediante la comprobación de un token. En la gestión de acceso, los servidores utilizan la autenticación por token para comprobar la identidad de un usuario, una API, un ordenador u otro servidor.



Un token es un elemento simbólico que expide una fuente de confianza y puede hacer referencia a un par de procesos diferentes:

Verificar la identidad mediante un token físico: Es un factor de autenticación muy habitual para iniciar sesión: se pide a los usuarios que presenten su token al iniciar sesión en una cuenta o en un dispositivo. Muy habitual en el sector bancario y de impuestos

Corroborar la identidad mediante un token web: Los tokens web son completamente digitales. Un token web es generado por un servidor y enviado a un cliente. El token se adjunta a cada solicitud del cliente para que el servidor conozca la identidad del cliente y sepa a qué datos puede acceder. Este tipo de autenticación basada en tokens es diferente de la autenticación basada en cookies, que es una forma similar de conseguir lo mismo.

¿Cómo funciona la autenticación con un token físico?

La autenticación mediante un token físico suele tener lugar durante el proceso de inicio de sesión del usuario. El usuario tiene que demostrar que posee un objeto que nadie más tiene. Puede demostrarlo introduciendo un código mostrado por el objeto, conectando el objeto a un dispositivo mediante USB, conectando el objeto mediante Bluetooth o por varios otros métodos. Al igual que introducir una contraseña demuestra que el usuario posee un conocimiento que nadie más tiene, el uso de un token demuestra que un usuario posee un elemento que solo él tiene.



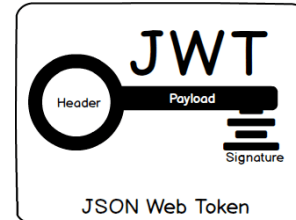
Se utilizan dos tipos de tokens para este tipo de autenticación: tokens blandos tokens duros.

Los **tokens blandos** implican introducir un código secreto o un mensaje enviado a un dispositivo para demostrar la posesión del mismo. Suele ser un código enviado a un teléfono inteligente a través de un mensaje de texto.

Lo **tokens duros** son elementos de hardware que el usuario conecta directamente a un ordenador o dispositivo móvil para iniciar sesión.

¿Cómo funciona la autenticación mediante token web?

Un token web es digital, no un objeto físico. Es un mensaje enviado desde un servidor a un cliente y que este almacena temporalmente. El cliente incluye una copia del token en las siguientes solicitudes enviadas al servidor para confirmar el estado de autenticación del cliente.



Mientras que la autenticación con tokens físicos verifica la identidad durante el proceso de inicio de sesión, los tokens web se emiten como resultado de un inicio de sesión con éxito. Mantienen activa la sesión iniciada.

Sin embargo, utilizar tokens web para las sesiones de usuario no siempre es lo mejor. Muchos desarrolladores son partidarios de utilizar cookies en su lugar. Los tokens web se pueden utilizar mejor para la autenticación de puntos finales de la API o para validar una conexión entre servidores, en lugar de entre el servidor y el cliente.

¿Qué es el Token web JSON (JWT)?

En desarrollo web, los "tokens web" hacen referencia casi siempre a los Tokens web JSON. El Token web JSON (JWT) es un estándar para crear tokens web firmados digitalmente que contienen datos de Notación de objetos de JavaScript (JSON). Un servidor crea un token que demuestra la identidad del cliente y se lo envía a este. JWT utiliza firmas digitales para demostrar que el token es legítimo.

Los JWT incluyen tres componentes:

Encabezado: el encabezado proporciona información sobre el JWT: qué tipo de token es y qué método se ha utilizado para firmarlo digitalmente.

Carga útil: cualquier dato JSON puede ir aquí. Las cargas útiles de JWT para la autenticación incluyen afirmaciones sobre la identidad del usuario en la carga útil. También pueden incluir información sobre los permisos del usuario, del servidor o del punto final de la API.

Firma digital: la firma utiliza criptografía para firmar el encabezado y la carga útil con una clave para asegurar que los datos que contienen sean legítimos. Pensemos que la firma digital es como el precinto del envase de un medicamento.

Autenticación basada en tokens (JWT) vs. Autenticación basada en cookies

En ocasiones, los JWT se utilizan para mantener a los usuarios autenticados una vez que han iniciado sesión en una aplicación web. No obstante, las cookies también pueden utilizarse para este fin.

Una cookie es un pequeño archivo de datos que un servidor envía a un cliente. Cuando un usuario se registra en una aplicación web, el servidor genera una cookie y la envía al dispositivo cliente (normalmente el ordenador o el teléfono inteligente del usuario). El dispositivo cliente almacena la

cookie en la caché del navegador e incluye una copia de la cookie en futuras solicitudes al servidor, parecido a como se pueden utilizar los JWT. Una vez el usuario cierra la sesión, el navegador elimina la cookie.

Las cookies tienen un tamaño de archivo mucho menor en comparación con los JWT, porque los JWT incluyen encabezados y firmas digitales además de la carga útil. En cambio, la cookie solo contiene la carga útil. Esto las hace mucho más eficientes en términos de rendimiento web y de ancho de banda.

Rendimiento web: una cookie se carga más rápido porque contiene menos información, al igual que una foto se descarga más rápido que un vídeo.

Ancho de banda: como las cookies son más pequeñas, reducen la cantidad total de datos que deben pasar por las redes entre el cliente y el servidor. Esto puede suponer un ahorro de costes para el operador de la aplicación web en comparación con los JWT.

Los JWT no están optimizados para el rendimiento porque incluyen firmas digitales, que garantizan que no se haya manipulado su contenido. Pero si una aplicación web utiliza HTTPS, una cookie debería tener defensas contra manipulaciones de todos modos. HTTPS encriptará y firmará la cookie junto con todos los demás datos HTTP que se intercambien entre el cliente y el servidor, y los atacantes no deberían poder falsificarla o interceptarla en tránsito, a menos que realicen un ataque en ruta.

Los JWT son más apropiados para las API y las conexiones de servidor a servidor. Estos usos no necesitan escalar tanto: una aplicación web puede tener un millón de usuarios, pero no va a tener un millón de conexiones API. Esto reduce el impacto potencial sobre el rendimiento y el ancho de banda.

Json Web Token

JWT (JSON Web Token) es un estándar que está dentro del documento RFC 7519. En este documento se define un mecanismo para poder propagar entre dos partes, y de forma segura, la identidad de un determinado usuario, además con una serie de claims o privilegios.

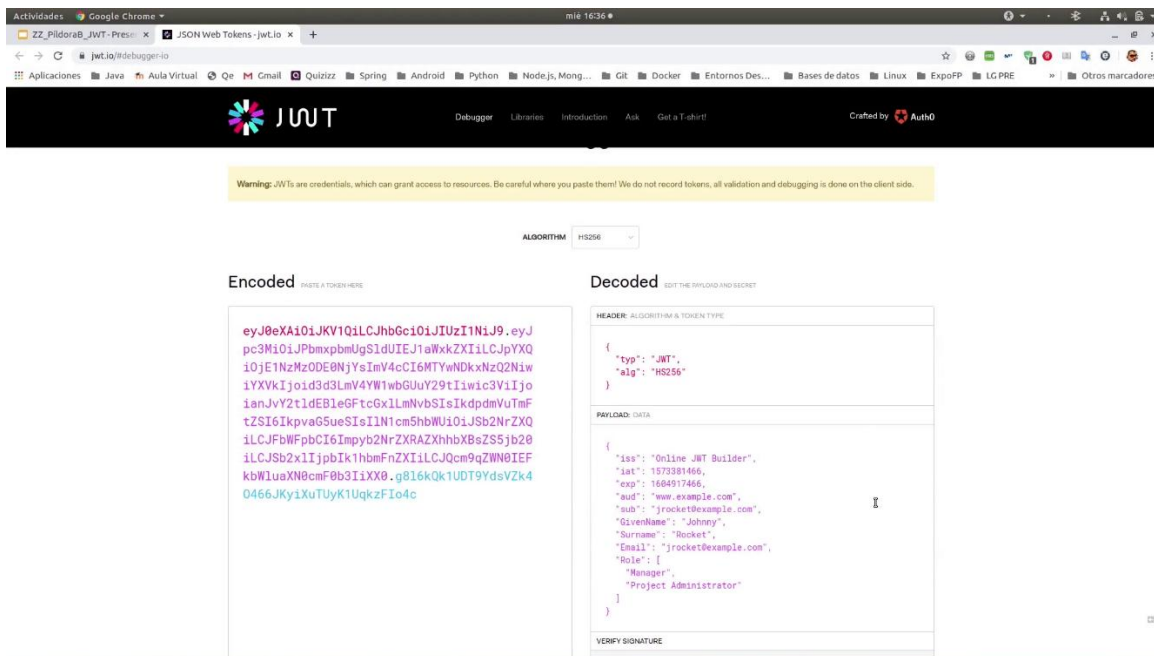
Estos privilegios están codificados en objetos de tipo JSON, que se incrustan dentro del payload o cuerpo de un mensaje que va firmado digitalmente.

Token JWT

En la práctica, se trata de una cadena de texto que tiene tres partes codificadas en Base64, cada una de ellas separadas por un punto, como la que vemos en la imagen siguiente:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.ikFGEvw-Du0f30vBaA742D_wqPA5BBHXgUY6wwqab1w
```

Podemos utilizar un [debugger online](#) para decodificar ese token y visualizar su contenido. Si accedemos al mismo y pegamos dentro el texto completo, se nos mostrará lo que contiene:



Podemos ver el contenido del token sin necesidad de saber la clave con la cual se ha generado, aunque no podremos validarlo sin la misma.

Como hemos dicho, un token tiene tres partes:

- **Header:** encabezado donde se indica, al menos, el algoritmo y el tipo de token, que en el caso del ejemplo anterior era el algoritmo HS256 y un token JWT.
- **Payload:** donde aparecen los datos de usuario y privilegios, así como toda la información que queramos añadir, todos los datos que creamos convenientes.
- **Signature:** una firma que nos permite verificar si el token es válido, y aquí es donde radica el quid de la cuestión, ya que si estamos tratando de hacer una comunicación segura entre partes y hemos visto que podemos coger cualquier token y ver su contenido con una herramienta sencilla, ¿dónde reside entonces la potencia de todo esto?

Firma de un token JWT

La firma se construye de tal forma que vamos a poder verificar que el remitente es quien dice ser, y que el mensaje no se ha modificado por el camino.

Se construye como el HMACSHA256, que son las siglas de Hash-Based Message Authentication Code (Código de Autenticación de Mensajes), cifrado además con el algoritmo SHA de 256 bits. Se aplica esa función a:

- **Codificación en Base64 de header.**
- **Codificación en Base64 de payload.**
- **Un secreto**, establecido por la aplicación.

De esta forma, si alguien modifica el token por el camino, por ejemplo, inyectando alguna credencial o algún dato malicioso, entonces podríamos verificar que la comprobación de la firma no es correcta, por lo que no podemos confiar en el token recibido y deberíamos denegar la solicitud de recursos que nos haya realizado, ya sea para obtener datos o modificarlos.

Si lo que estamos requiriendo es que el usuario esté autenticado, deberíamos denegar esa petición, por lo que siempre que trabajemos con JWT deberíamos verificar con esa firma si el token es válido o no lo es.

Token JWT seguro

Aunque el algoritmo nos permita verificar la firma, y, por tanto, confiar o no, tanto el encabezado como el cuerpo si llevan muchos datos van en abierto, ya que Base64 no es un cifrado, es simplemente una codificación que es muy fácilmente decodificable.

JWT nos invita siempre a que la comunicación entre partes se realice con HTTPS para encriptar el tráfico, de forma que, si alguien lo interceptara, el propio tráfico a través de HTTP sobre esos sockets SSL, cifrara toda la comunicación, la del token y todo lo demás, y así añadir esa posibilidad de seguridad.

De hecho, **siempre deberíamos utilizar HTTPS y un servidor con certificado** para hacer el despliegue

Ciclo de vida de un Token JWT

Vamos a ver ahora el ciclo de vida de un token JWT, si lo queremos utilizar en el marco de un proceso de autenticación.

Como hemos visto, JWT no es un estándar de autenticación, sino que simplemente un estándar que nos permite hacer una comunicación entre dos partes de identidad de usuario. Con este proceso, además, podríamos implementar la autenticación de usuario de una manera que fuera relativamente segura.



Comenzaríamos desde el cliente, haciendo una petición POST para enviar el usuario y contraseña, y realizar el proceso de login.

Se comprobaría que ese usuario y su contraseña son correctos, y de serlos, generar el token JWT para devolverlo al usuario.

A partir de ahí la aplicación cliente, con ese token, haría peticiones solicitando recursos, siempre con ese token JWT dentro de un encabezado, que sería `Authorization: Bearer XXXXXXXX`, siendo Bearer el tipo de prefijo seguido de todo el contenido del token.

En el servidor se comprobaría el token mediante la firma, para verificar que el token es seguro, y, por tanto, podemos confiar en el usuario.

Dentro del cuerpo del token, además, tenemos los datos de quién es el usuario que ha realizado esa petición, porque podemos contener en el payload todos los datos de usuario que queramos.

Tras verificar que el token es correcto y saber quién es el que ha hecho la petición, podemos aplicar entonces el mecanismo de control de acceso, saber si puede acceder o no, y si es así, responder con el recurso protegido, de manera que lo podría recibir de una forma correcta.

De esta forma podríamos **implementar el proceso de autenticación**, y hacerlo, además, con estos JSON Web Token.

INTEGRACION CON NODE JS.

Para integrar jsonwebtoken en nuestros proyectos de node, haremos uso de la librería: <https://www.npmjs.com/package/jsonwebtoken>, haremos un ejemplo a continuación pero como siempre les recomiendo leer la documentación.

Para la integración con Nodejs vamos a tener en cuenta el ciclo de vida de un token JWT expuesta en la imagen anterior, así, que lo primero que debemos hacer es crear el token en el momento en que nuestros usuarios se loguean en la API.

A continuación, les expondré una **forma** detallada de hacerlo basados en la documentación de la librería jsonwebtoken, y como todo en programación, **recuerden que no es la única**. Además, recuerden también que los nombres de los archivos, variables y funciones las pueden llamar a su antojo.

GENERACION DEL TOKEN

Función generarJWT: Crearemos un archivo en nuestra carpeta middlewares, al que llamaremos validar-jwt.js y nos crearemos la función generarJWT(), que se encargara de crear nuestro token. El contenido de la función seria:

```
const generarJWT = (uid) => {  
  return new Promise((resolve, reject) => {  
    const payload = { uid };  
    jwt.sign(payload, process.env.SECRETORPRIVATEKEY, {  
      expiresIn: "4h"//4h  
    }, (err, token) => {
```

```

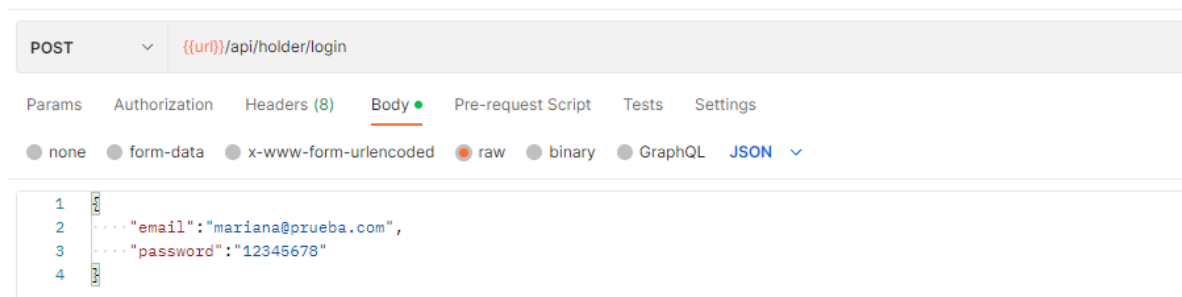
    if (err) {
      console.log(err);
      reject("No se pudo generar el token")
    } else {
      resolve(token)
    }
  })
})
}

```

Analizando el código anterior, observamos que:

- Como parámetro de la función generarJWT, recibirá la información que deseamos tokenizar.
- La firma del token (función sign), requiere de dos parámetros, el primero la información a tokenizar, y el segundo una clave para firmar nuestro token.
- Noten que dicha clave ha sido guardada en nuestro archivo .env.
- La creación del token, tiene un parámetro expiresIn, en el cual indicamos la duración del mismo. Se puede especificar en segundos, horas, etc. Para saber más deben consultar la documentación de la librería.
- Como podemos observar la librería jsonwebtoken utilizada, se encarga de todo, lo único que hacemos es pasarle los parámetros necesarios.
- En resumen, nuestra función generarJWT, es una promesa que recibe la información a tokenizar y devuelve el token generado.

Entonces, el siguiente paso es: que sucede después que llamamos nuestra ruta login?



Bien, ahora al momento de loguearse simplemente llamamos a la función generarJWT (eso si enviando la información a tokenizar, que para este caso es el id de un usuario), y capturamos el token que nos regresa

```
const token = await generarJWT(holder.id);
```

Veamos cómo podría ser el código completo de nuestro Login

```

login: async (req, res) => {
  const { email, password } = req.body;
  try {
    const holder = await Holder.findOne({ email })
    if (!holder) {
      return res.status(400).json({
        msg: "Holder / Password no son correctos"
      })
    }
  }
}

```



```

    })
  }
  if (holder.estado === 0) {
    return res.status(400).json({
      msg: "Holder Inactivo"
    })
  }
  const validPassword = bcryptjs.compareSync(password, holder.password);
  if (!validPassword) {
    return res.status(400).json({
      msg: "Holder / Password no son correctos"
    })
  }
  const token = await generarJWT(holder.id);

  res.json({
    holder,
    token
  })
} catch (error) {
  return res.status(500).json({
    msg: "Hable con el WebMaster"
  })
}
},

```

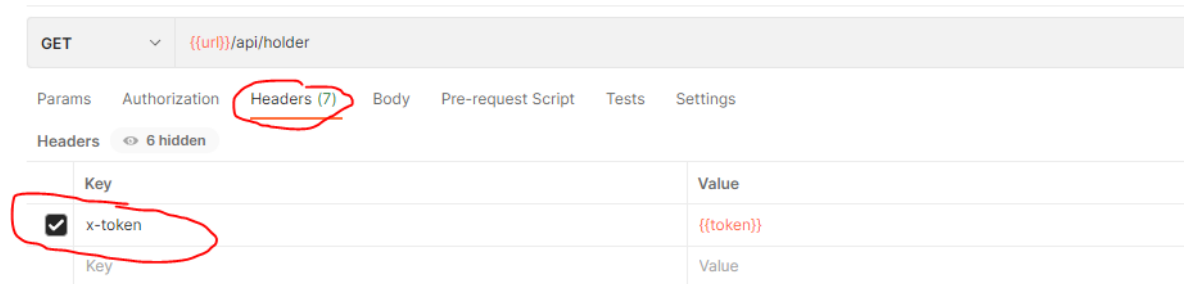
Analizando el código anterior:

- En primera instancia se valida, tanto el usuario, la clave y el estado del usuario para saber si es un usuario válido en nuestra base de datos.
- Como siguiente paso, se genera el token.
- Por último respondemos la llamada HTTP, con la información del usuario logueado y el token generado, para que pueda ser recuperado en el frontend.

Con esto cubrimos las 3 primeras partes del ciclo de vida de un json web token, ahora miremos que como proteger nuestras rutas con dicho token.

PROTECCION DE LLAMADAS CON UN TOKEN

Llamadas: Ahora las llamadas a rutas que estén protegidas por un token, deben incluir en su encabezado el token a validar. ej



Función validarJWT: en el mismo archivo validar-JWT.js, creado anteriormente, crearemos una función llamada validarJWT(), la cual se encargara de validar la autenticidad de nuestro token. El contenido de la función seria:

```
const validarJWT = async (req, res, next) => {
  const token = req.header("x-token");
  if (!token) {
    return res.status(401).json({
      msg: "No hay token en la petición"
    })
  }
  try {
    const { uid } = jwt.verify(token, process.env.SECRETORPRIVATEKEY)

    let usuario = await Holder.findById(uid);

    if (!usuario) {
      return res.status(401).json({
        msg: "Token no válido " //- usuario no existe DB
      })
    }
    if (usuario.estado == 0) {
      return res.status(401).json({
        msg: "Token no válido " //- usuario con estado: false
      })
    }

    req.usuario=usuario    //- pasamos el usuario al request para poder usarlo en cualquier
                          // petición

    next();
  } catch (error) {
    res.status(401).json({
      msg: "Token no valido"
    })
  }
}
```

Analizando el código anterior:

- Se destructure el token enviado a través del **header** en la llamada. Para nuestro caso se llama "x-token", pero pueden llamarlo como quieran.
- Después de validar que exista el token, se lo pasamos a la función verify de la librería jsonwebtoken para la comprobación de veracidad del mismo.
- La función verify nos devuelve un objeto con la información almacenada en el token. Si lo recuerdan, y revisan más arriba, solo habíamos tokenizado un objeto con el **uid** de un usuario, que es lo mismo que estructuramos ahora.
- Por ultimo tomamos es **uid** devuelto por el token, y verificamos que sea un usuario valido, es decir, para nuestro caso validar que exista en la base de datos y que su estado sea activo. Aquí también podrían validar de ser necesario los roles de los usuarios, pagos de membresías, etc.

Proteger la ruta con el token: Ya creamos la función que valida el token, ahora habrá que llamarla en los middlewares de toda ruta que deseemos proteger por un jsonwebtoken. Ejs.

```
56 router.put('/:id',[
57     validarJWT,
58     check('id', 'No es un ID válido').isMongoId(),
59     check('id').custom(helpersHolder.existeHolderById),
60     check("document").custom(helpersHolder.existeNumDocumento),
61     //check('email').custom( helpersHolder.existeEmail ),
62     validarCampos
63 ],holdersHttp.holderPut);
64
65 router.put('/activate/:id',[
66     validarJWT,
67     validarRol('ADMIN'),
68     check('id', 'No es un ID válido').isMongoId(),
69     check('id').custom(helpersHolder.existeHolderById),
70     validarCampos
71 ],holdersHttp.holderPutActivate);
72
```